

CS-4110 HPC with GPUs

Dr Imran Ashraf
Associate Professor
CS, NUCES-FAST, Islamabad
C-205-E

This week

- Unified Memory
- Coalesced Global Memory Access
- Shared Memory Bank Conflicts

Traditional GPU Memory Management

- Separate memory spaces for CPU (RAM) & GPU (VRAM)
- Requires manual memory allocation & data transfer using cudaMemcpy

Challenges

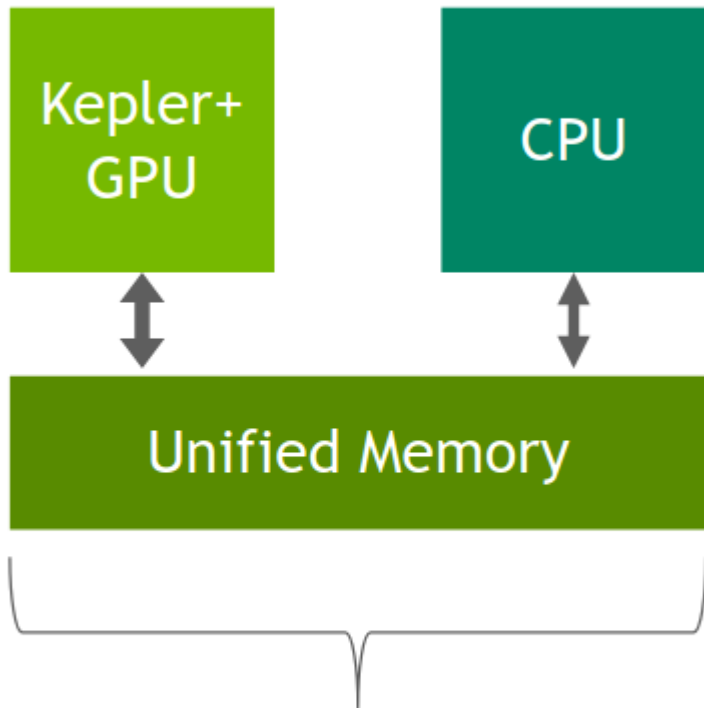
- Increased programming effort
- Performance bottlenecks due to unnecessary memory copies

Unified Memory Model

- **Unified Memory**
 - A single memory space accessible by both CPU and GPU.
 - Eliminates explicit `cudaMemcpy()` operations.
- **Why Unified Memory?**
 - Simplifies programming
 - Reduces memory management complexity
 - Enables dynamic memory migration between CPU & GPU.

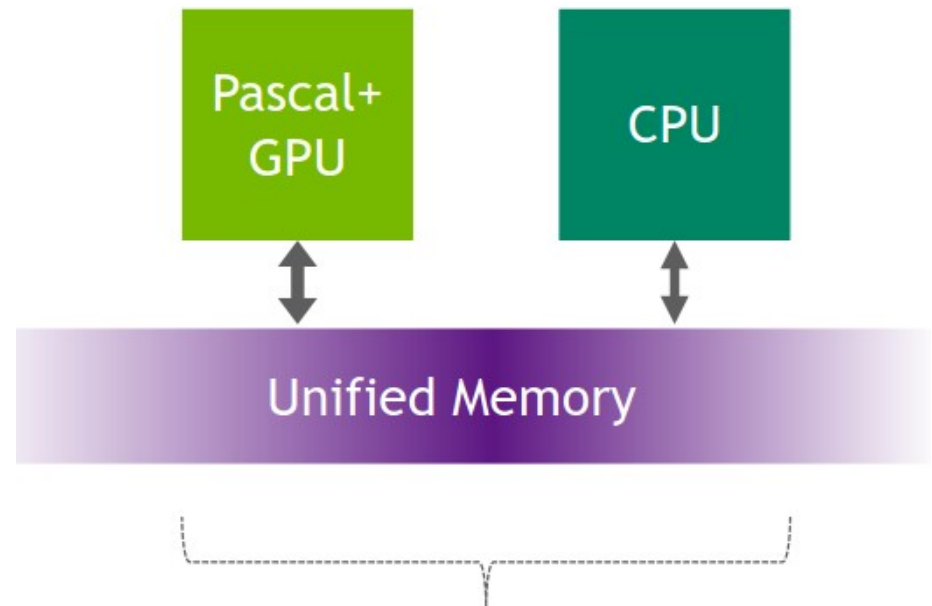
Unified Memory Model

CUDA 6+



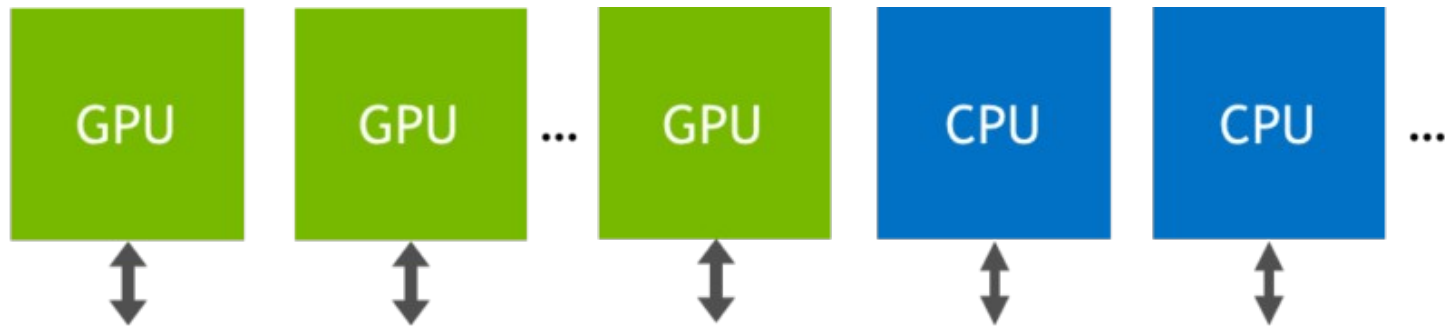
Allocate Up To
GPU Memory Size

CUDA 8+



Allocate Beyond
GPU Memory Size

Unified Memory Model

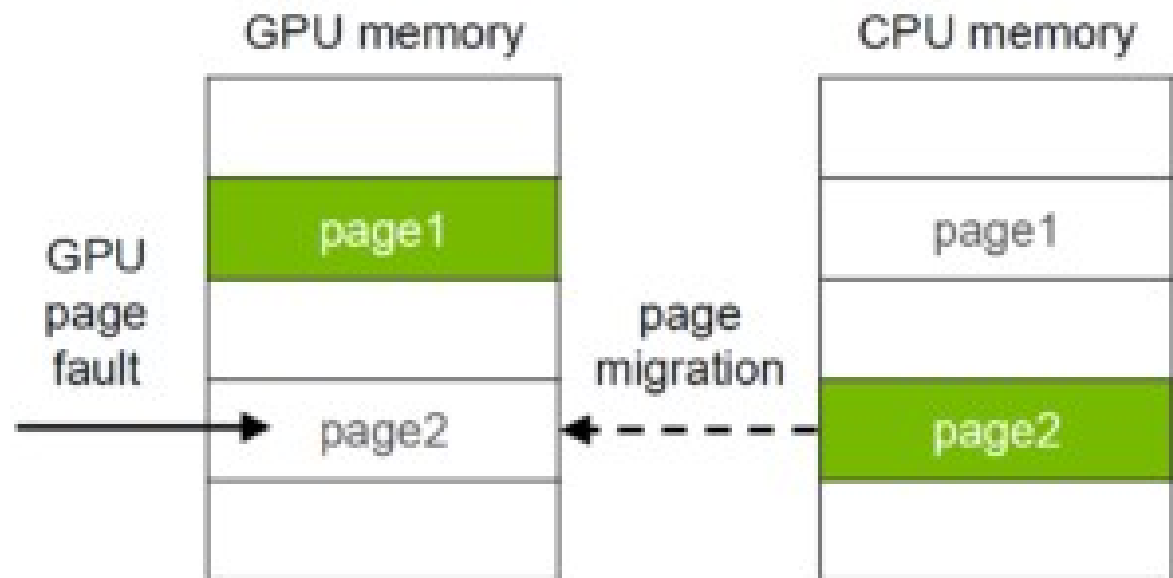


Unified Memory Working

- Uses Unified Virtual Addressing (UVA) to create a single address space.
- On-Demand Page Migration: Moves memory pages automatically based on access.
- Lazy Allocation: Allocates memory only when accessed.

Page Migration

1. Allocate new pages on the GPU;
2. Unmap old pages on the CPU;
3. Copy data from the CPU to the GPU;
4. Map new pages on the GPU;
5. Free old CPU pages.



Unified Memory: Example

```
int *data;
```

```
// Unified Memory Allocation  
cudaMallocManaged(&data, size);
```

```
// GPU Access  
kernel<<<blocks, threads>>>(data);  
cudaDeviceSynchronize();
```

```
cudaFree(data);
```

Benefits of Unified Memory

- Page Migration engine allows GPU threads to fault on non-resident memory accesses so the system can migrate pages on demand from anywhere in the system to the GPU's memory
- Unified Memory transparently enables oversubscribing GPU memory, enabling out-of-core computations
- GPUs support system-wide atomic memory operations
- Demand paging can be particularly beneficial to applications that access data with a sparse pattern.

Unified Memory Use-case

- Deep Learning & AI: Handles large datasets exceeding GPU VRAM.
- Scientific Computing: Allows massive simulations to run across CPU & GPU.
- Big Data Analytics: Reduces memory management complexity in GPU databases.
- HPC & Supercomputing: Enables efficient memory sharing across heterogeneous systems.

Unified Memory Performance

- kernel call runs much slower
- Each page fault triggers service overhead. Relying on page faults to move large amounts of data, page-by-page, with overhead on each page, is inefficient.
- For bulk movement, a single “memcpy-like” operation is efficient

Pre-fetching

```
int device = -1;
cudaGetDevice(&device);

// Prefetch the data to the GPU
cudaMemPrefetchAsync(x, N*sizeof(float), device, NULL);
cudaMemPrefetchAsync(y, N*sizeof(float), device, NULL);

// Run kernel on 1M elements on the GPU
int blockSize = 256;
int numBlocks = (N + blockSize - 1) / blockSize;
saxpy<<<numBlocks, blockSize>>>(N, 1.0f, x, y);
```

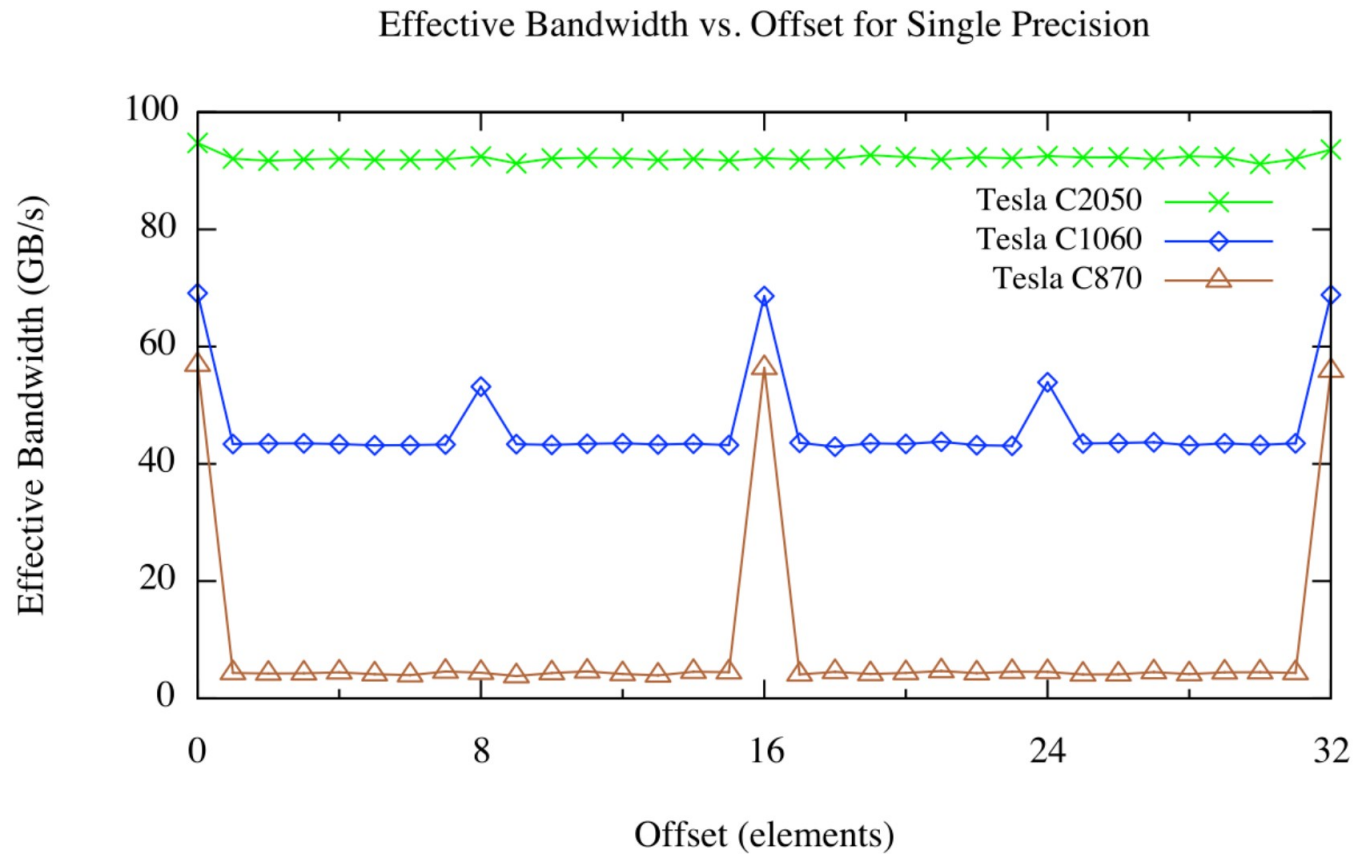
Global Memory Access Patterns

- Mis-aligned access
- Strided accesses

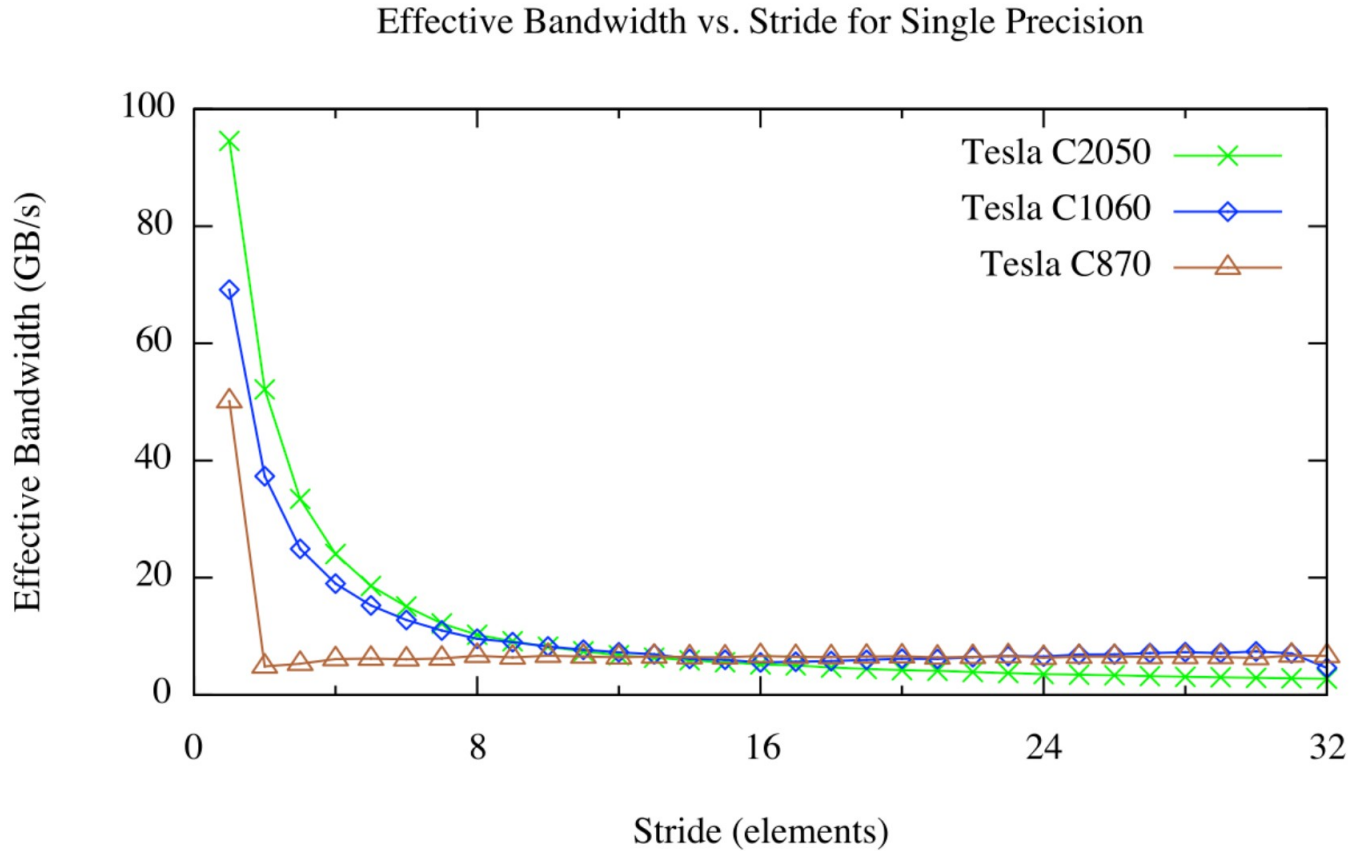
```
template
__global__ void offset(T* a, int s)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x + s;
    a[i] = a[i] + 1;
}

template
__global__ void stride(T* a, int s)
{
    int i = (blockDim.x * blockIdx.x + threadIdx.x) * s;
    a[i] = a[i] + 1;
}
```

Misaligned Data Access



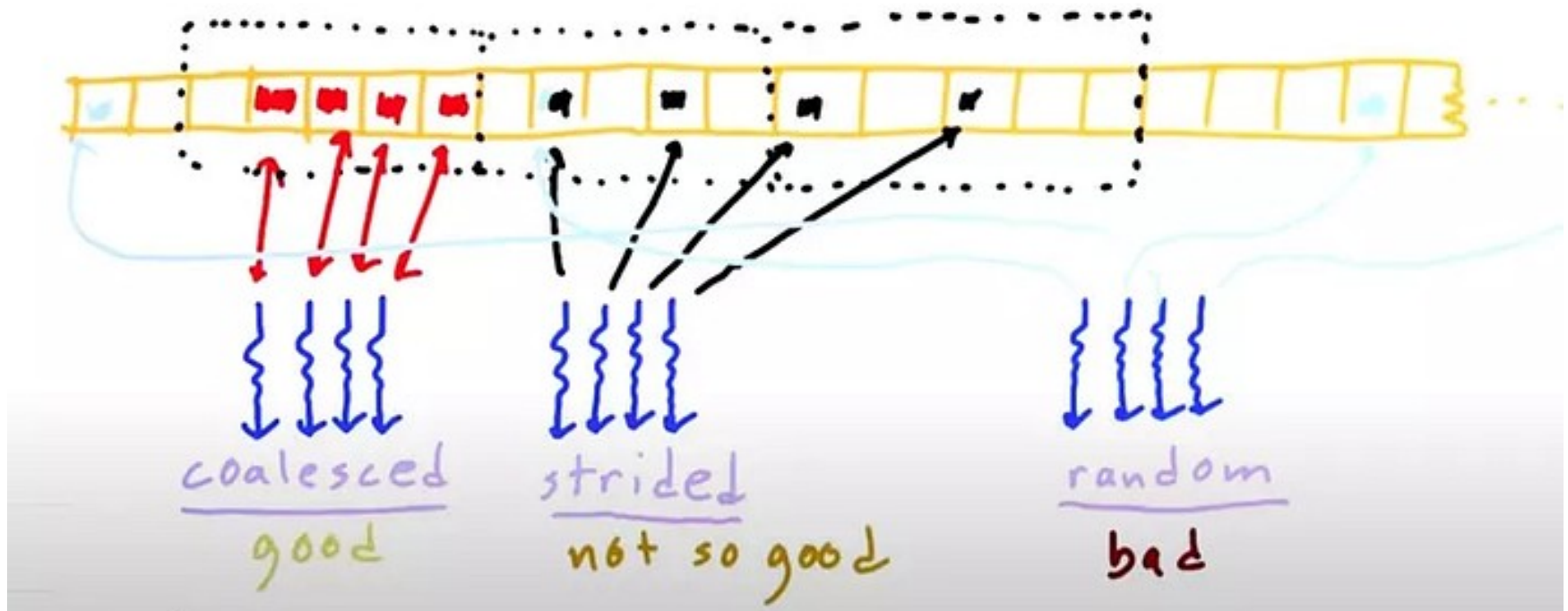
Strided Memory Access



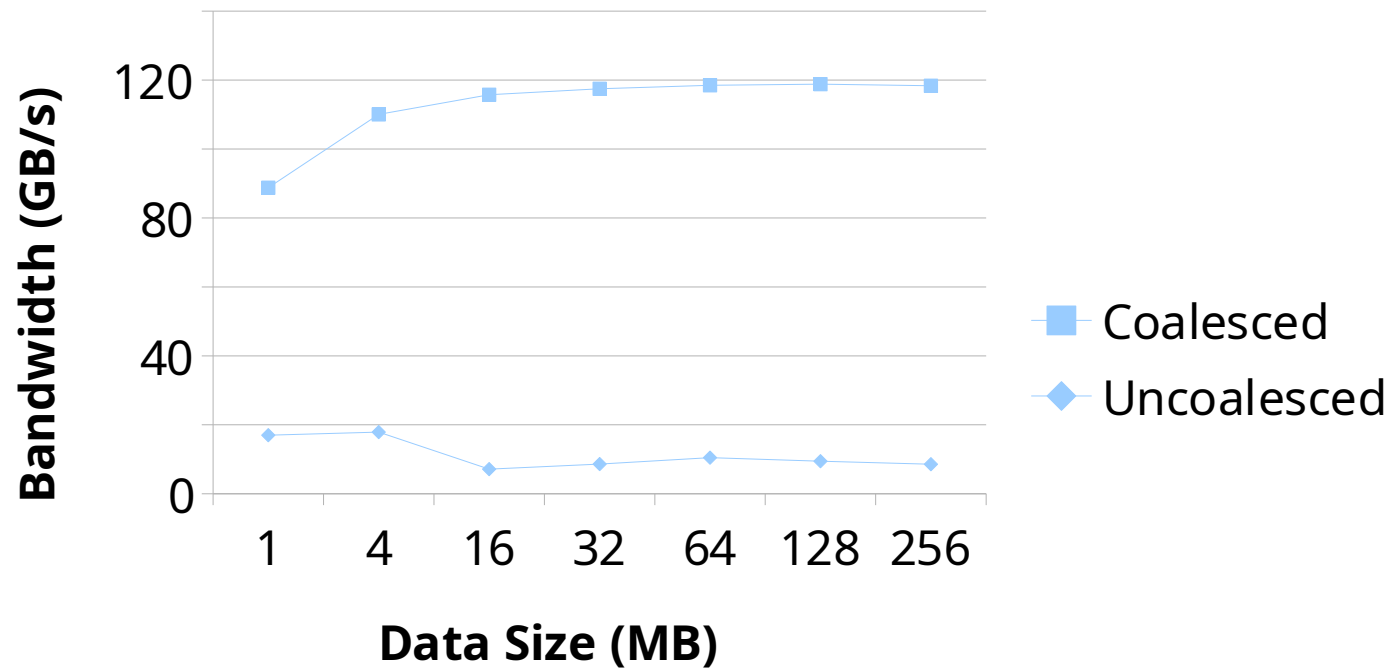
Coalesced Global Memory Access

- To fully utilize the bus, GPUs combine the accesses of multiple threads into fewer requests when possible
- When GPU threads access data in the same 32-byte range, multiple accesses are combined so that each range is accessed once
 - Combining accesses is called coalescing

Coalesced Memory Access

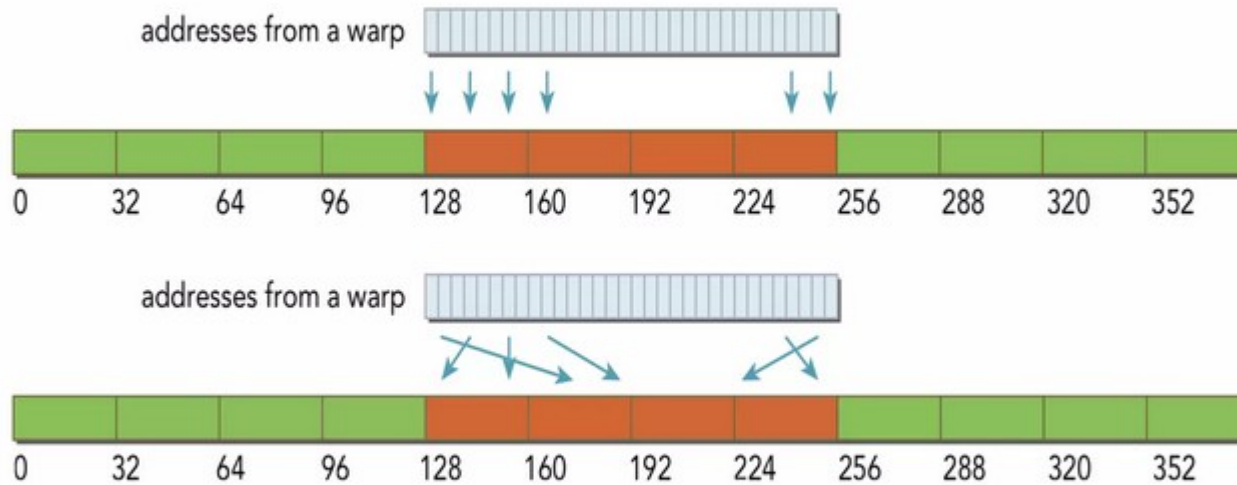


Coalesced vs Non-coalesced



GM Access Patterns

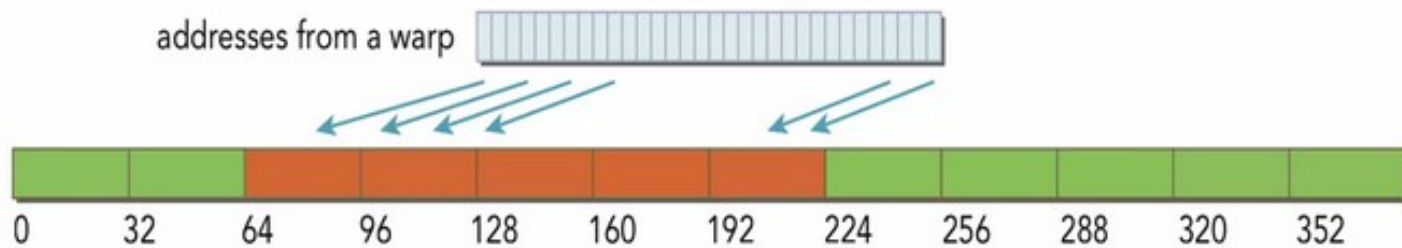
- Aligned and Coalesced Memory Access (*w/o L1 cache*)



- With 32-byte transactions, four transactions are required and all of the loaded bytes are used

GM Access Patterns

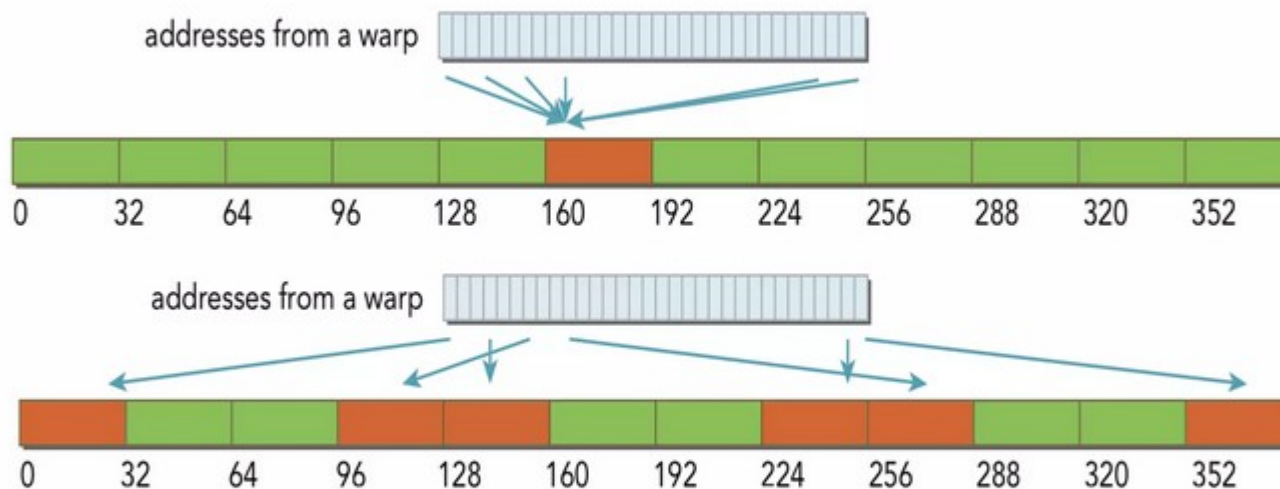
- Misaligned and Coalesced Memory Access (*w/o L1 cache*)



- With 32-byte transactions, extra memory transactions are still required to load all requested bytes but the number of wasted bytes is likely reduced, compared to 128-byte transactions.

GM Access Patterns

- Misaligned and Uncoalesced Memory Access (w/o L1 cache)



- With uncoalesced loads, more bytes loaded than requested but better efficiency than with 128-byte transactions

Coalesced access: CUDA

```
__global__ void coalesced_access(int *d_data) {  
    int idx = threadIdx.x + blockIdx.x * blockDim.x;  
  
    if (idx < N) {  
        // Each thread accesses consecutive elements  
        d_data[idx] += 5;  
    }  
}
```

Non-Coalesced access: CUDA

```
__global__ void non_coalesced_access(int *d_data) {  
    int idx = threadIdx.x + blockIdx.x * blockDim.x;  
  
    if (idx < N) {  
        d_data[idx * 2] += 5; // Strided access  
    }  
}
```


Quiz Coalesced 1

```
__global__ void vecAdd(float *A, float *B, float *C, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

Quiz Coalesced 2

```
__global__ void transMat(float *out, const float *in) {  
    int x = blockIdx.x * blockDim.x + threadIdx.x;  
    int y = blockIdx.y * blockDim.y + threadIdx.y;  
    if (x < N && y < N) {  
        out[x * N + y] = in[y * N + x];  
    }  
}
```

Quiz Coalesced 3

```
__global__ void mMul(float *A, float *B, float *C, int n) {  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
    if (row < n && col < n) {  
        float sum = 0.0f;  
        for (int k = 0; k < n; k++) {  
            sum += A[row * n + k] * B[k * n + col];  
        }  
        C[row * n + col] = sum;  
    }  
}
```

Quiz Coalesced 3

```
__global__ void mMul(float *A, float *B, float *C, int n) {
```

```
....
```

```
    for (int k = 0; k < n; k++) {
```

```
        // A is accessed row-wise (coalesced)
```

```
        // B is accessed column-wise (UNCOALESCED)
```

```
        sum += A[row * n + k] * B[k * n + col];
```

```
    }
```

```
    // A is accessed row-wise (coalesced)
```

```
    C[row * n + col] = sum;
```

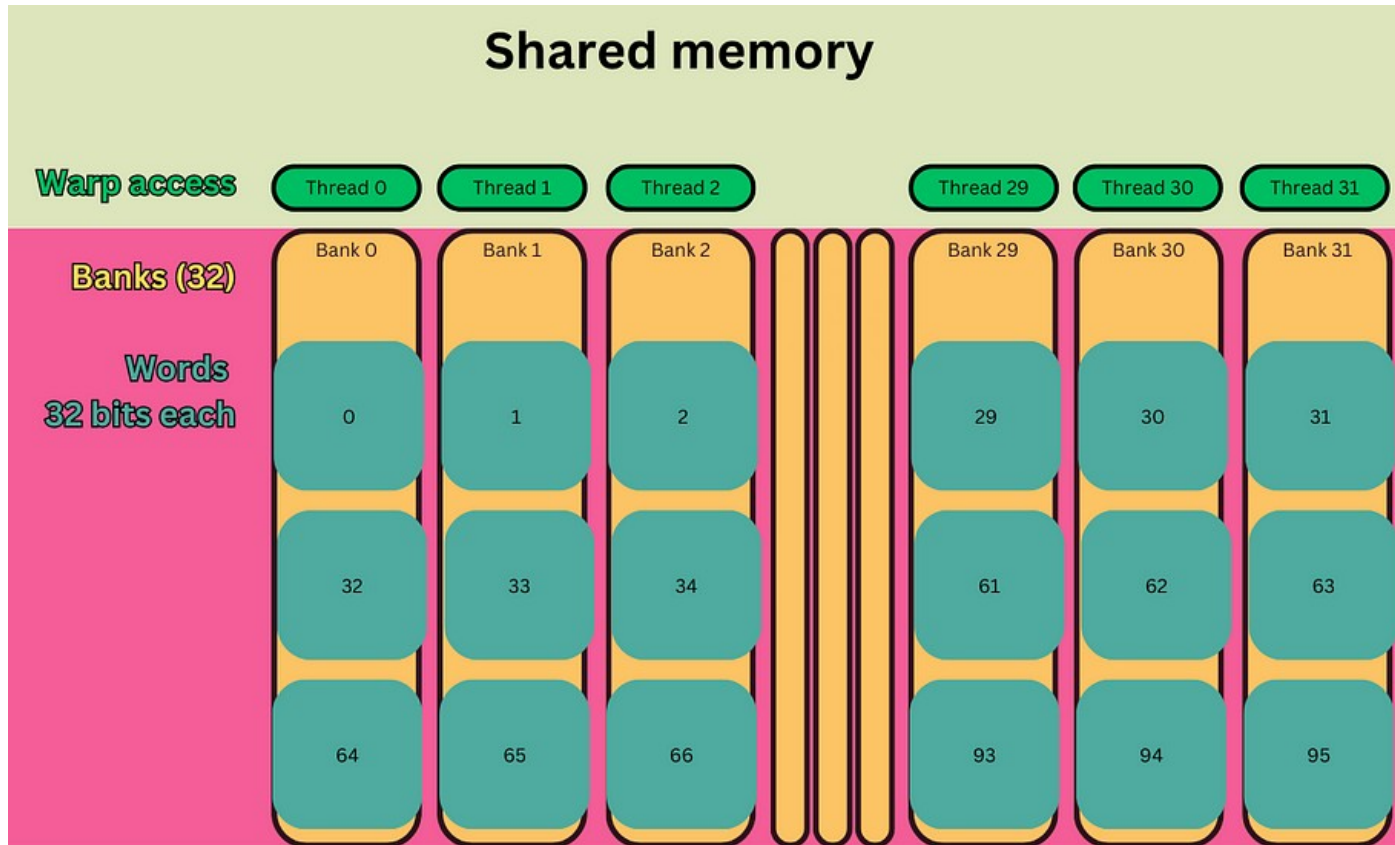
```
...
```

Shared Memory Bank Conflicts

Shared Memory Banks

- Shared memory in CUDA consists of 32 banks, organized such that successive 32-bit words map to successive banks. Each bank has a bandwidth of 32 bits per clock cycle.
- When a thread attempts to access shared memory, the access pattern across the threads in the warp can impact memory performance due to bank conflicts.

Shared Memory Banks



Shared Memory Bank Conflict

- A bank conflict can occur only in the same warp
- A bank conflict occurs when multiple threads within the same warp attempt to access memory locations that belong to the same memory bank simultaneously.
- This can lead to serialization of memory accesses, reducing memory bandwidth utilization and potentially slowing down the kernel's performance.

Shared Memory Bank Conflict

Example:

Warps size = Shared Memory banks count = 8

Data size = [2x8]

If each thread attempts access items by column *in the matrix* (i.e., warp0 => [0, 8]), it will lead to a Bank Conflict.

A matrix layout:

[[0, 1, 2, 3, 4, 5, 6, 7]

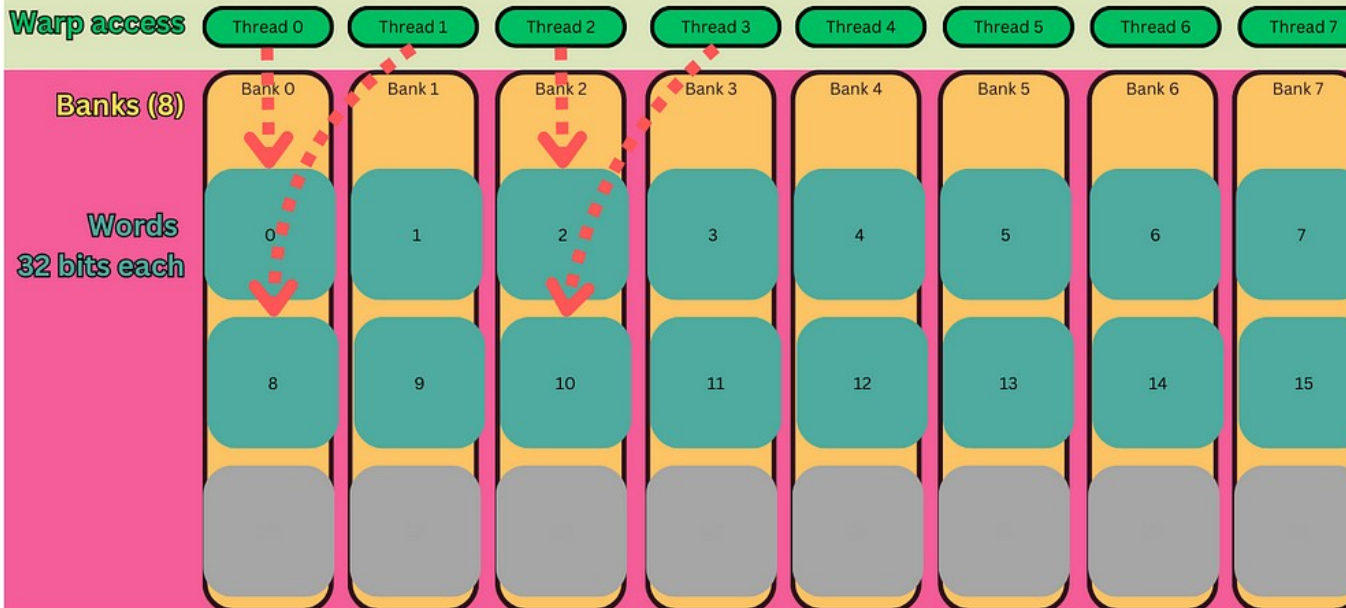
[8, 9, 10, 11, 12, 13, 14, 15]]

An address calculation formula:

$index = x + y * width$

- width = 8

Warp access

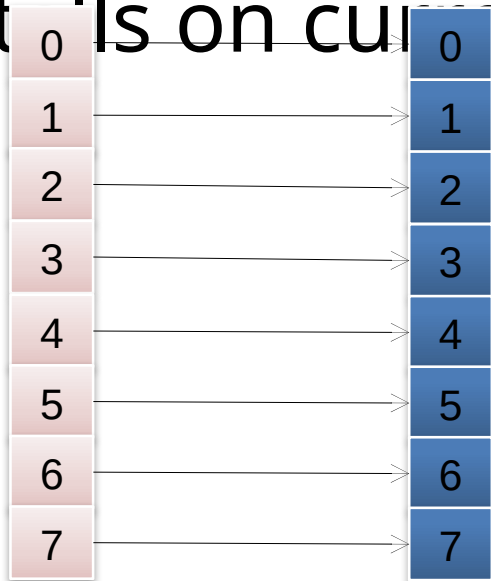


Banks (8)

Words
32 bits each

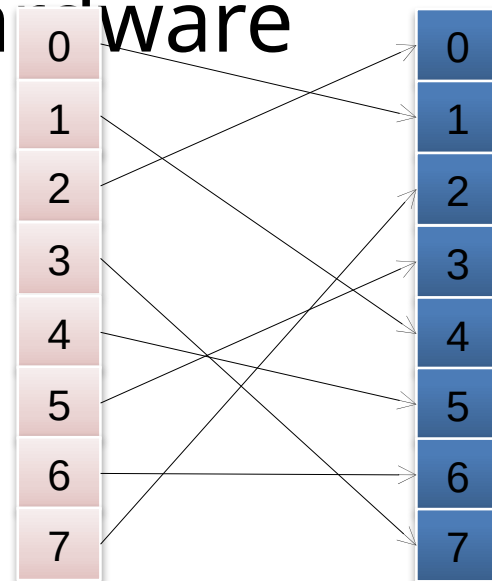
SM Bank Conflicts

- If there are no bank conflicts, each bank can return an element without any delays
- Both patterns will complete without stalls on current GPU hardware



Thread

Memory Bank

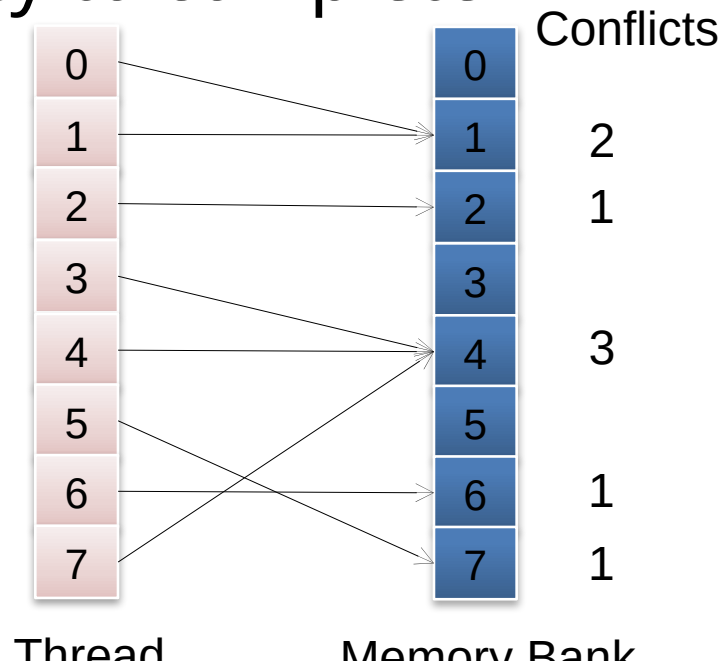


Thread

Memory Bank

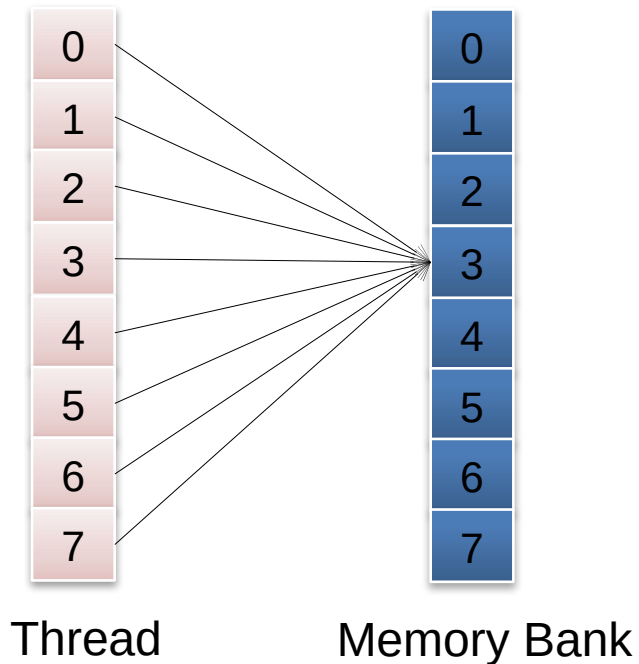
SM Bank Conflicts

- If multiple accesses occur to the same bank, then the bank with the most conflicts will determine the latency
- The following pattern will take 3 times the access latency to complete



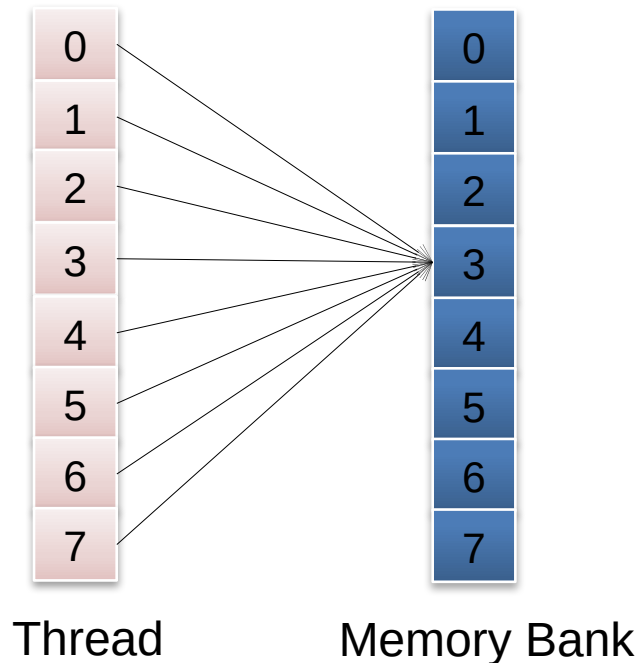
SM Bank Conflicts

- What about this access pattern?



SM Bank Conflicts

- If all accesses are to the same address, then the bank can perform a broadcast and no delay is incurred



Useful Reading

- <https://lwn.net/Articles/250967/>
- <https://people.cs.clemson.edu/~dhouse/courses/405/papers/optimize.pdf>